

Logic and Shift/Rotate Instructions

Computer Engineering 1

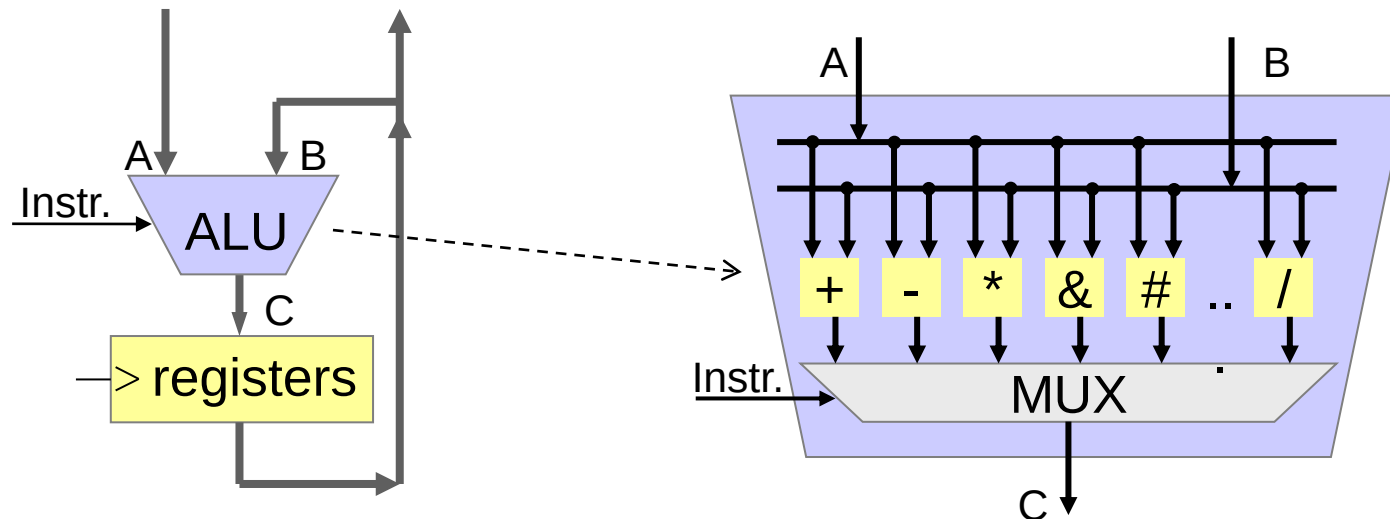
Motivation

- **So far, operands were bytes, half-words or words**
 - How can we work on a unit as small as a bit?
 - How can we look at a bit or a pattern of bits within a larger unit?
 - How can we modify a particular bit or a pattern of bits without affecting other bits?
- **Logic operations**
 - Help deal efficiently with elements at bit level
 - Enable the implementation of algorithms where Boolean operations are needed
- **Like in HW, shift operations can enable**
 - Division and multiplication
 - Test of certain bits
 - Communication, ...

Motivation

– Instructions to process data in ALU

- **arithmetic** Addition, Subtraction, Increment, Decrement, Multiplication, Division
- **logic** NOT, AND, OR, XOR
- **shift** Left/right shift. Fill up with 0 or MSB
- **rotate** Cyclic left/right shift: What falls out enters on the other side.



Agenda

- **Logic Instructions**
 - Bit manipulations
- **Shift/Rotate Instructions**
- **Multiplication with a Constant**

Learning Objectives

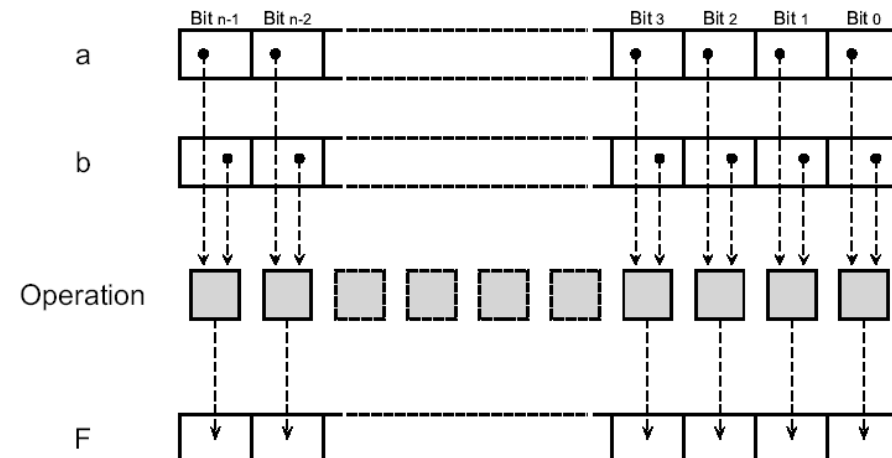
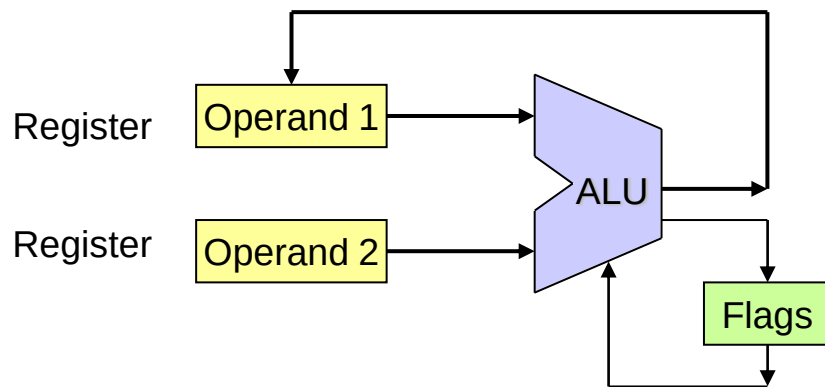
At the end of this lesson you will be able

- to enumerate and to apply the ARM instructions for logic as well as for shift/rotate operations
- to determine (with the help of documents) the state of the ARM Flags (N, Z, C, V) after the execution of an instruction
- to understand and interpret ARM assembly programs with logic and shift/rotate operations
- to explain bit manipulations based on examples
- to set, clear or invert one or several bits in a bit pattern
- to implement a multiplication with a constant in assembly using shift and add instructions

Logic Instructions Overview

Mnemonic	Instruction	Function	C-Operator
ands	Bitwise AND	Rdn & Rm	$a \& b$
bics	Bit Clear	Rdn & !Rm	$a \& \sim b$
eors	Exclusive OR	Rdn \$ Rm	$a \wedge b$
mvns	Bitwise NOT	!Rm	$\sim a$
orrs	Bitwise OR	Rdn # Rm	$a \mid b$

flags N = result<31>¹
 Z = 1 if result = 0
 Z = 0 otherwise
 C and V unchanged



¹ i.e. N is equal to bit 31 of the result
 MSB shows the sign of the result

Logic Instructions

Bitwise Operations

- All these operations affect the flags:
 - N and Z according to result
 - C and V unchanged
- Only low registers

mvns rd, rm *T1*
 rd, rm ∈ { r0..r7 }

010000	1111	mmm	ddd
--------	------	-----	-----

N Z

rd = ~rm;

orrs rd, rm *T1*
 rd, rm ∈ { r0..r7 }

010000	1100	mmm	ddd
--------	------	-----	-----

N Z

rd = rd | rm;

ands rd, rm *T1*
 rd, rm ∈ { r0..r7 }

010000	0000	mmm	ddd
--------	------	-----	-----

N Z

rd = rd & rm;

bics rd, rm *T1*
 rd, rm ∈ { r0..r7 }

010000	1110	mmm	ddd
--------	------	-----	-----

N Z

rd = rd & ~rm;

eors rd, rm *T1*
 rd, rm ∈ { r0..r7 }

010000	0001	mmm	ddd
--------	------	-----	-----

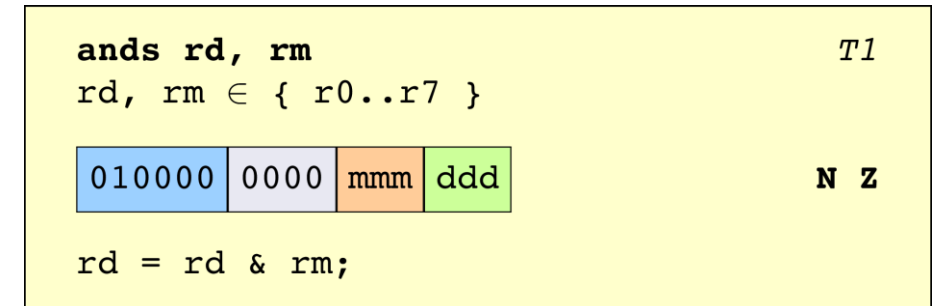
N Z

rd = rd ^ rm;

Logic Instructions

Example

- Update flags N and Z
- Only low registers possible!



00000002	4011	ands	r1,r1,r2	; r1 = r1 AND r2
00000004	4011	ands	r1,r2	; the same (dest = r1)
00000006	4337	orrs	r7,r7,r6	; r7 = r7 OR r6
00000008	4063	eors	r3,r3,r4	; r3 = r3 EXOR r8
0000000A	4388	bics	r0,r0,r1	; r0 = r0 AND NOT(r1)
0000000C	43D1	mvns	r1,r2	; r1 = NOT(r2)

Bit Manipulations

Bit Manipulations (Cortex-M0)

- Clear bits, e.g. clear bits 5 and 1 in register r1
- Set bits, e.g. set bits 6 und 3 in register r1
- Invert bits, e.g. invert bits 4, 3 and 2 in register r1
- What happens to the other bits?
- Compute the value of r1 after execution of all lines assuming that r1 contains 0xAF08'24A3 at the start.
- What are the values of the flags after execution?

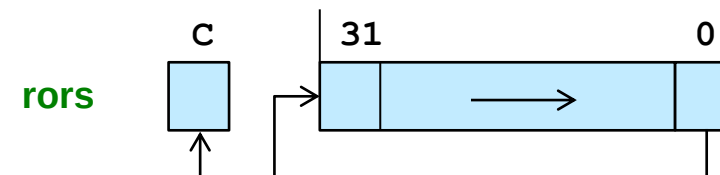
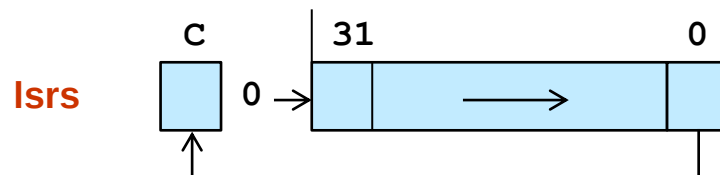
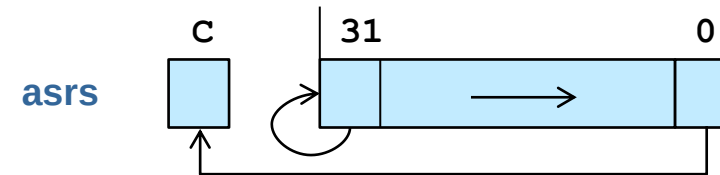
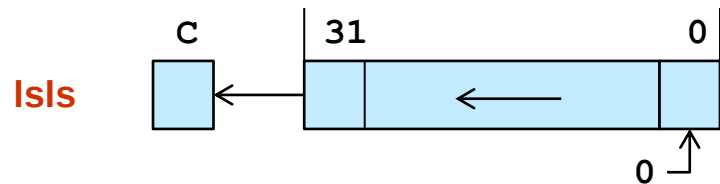
```
movs    r2,#0x22          ;00100010b
bics    r1,r1,r2
```

```
movs    r2,#0x48          ;01001000b
orrs    r1,r1,r2
```

```
movs    r2,#0x1C          ;00011100b
eors    r1,r1,r2
```

Shift / Rotate Instructions Overview

Mnemonic	Instruction	Function
lsls	Logical Shift Left	$2^n \cdot Rn$ 0 → LSB
lsrs	Logical Shift Right	$2^{-n} \cdot Rn$ 0 → MSB
asrs	Arithmetic Shift Right	$2^{-n} \cdot \pm A$ MSB → MSB
rors	Rotate Right	LSB → MSB



Note: rotate left does not exist

Shift / Rotate Instructions

Opcodes (register)

- Low registers only
- Flags affected

flags N = result<31>¹
 Z = 1 if result = 0
 Z = 0 otherwise
 C = last bit shifted out
 V unchanged

lsls rd, rm T1
 rd, rm ∈ { r0..r7 }
 010000 0010 mmm ddd N Z C
 rd = rd << rm;

lsrs rd, rm T1
 rd, rm ∈ { r0..r7 }
 010000 0011 mmm ddd N Z C
 rd = rd >> rm;

asrs rd, <rm>, #imm5 T1
 rd, rm ∈ { r0..r7 }
 imm5 ∈ { 0..31 }
 000 10 iiiii mmm ddd
 rd = extend_sign(rm >> imm5);
 or rd = extend_sign(rd >> imm5);
 (rd = rm / 2^{imm5})

rors rd, rm T1
 rd, rm ∈ { r0..r7 }
 010000 0111 mmm ddd N Z C
 rd = rd >> rm | rd << (32 - rm);
 (rotate right)

Shift / Rotate Instructions

Opcodes (immediate 0 – 31d)

- low registers only
- rors (immediate) not supported
- lsrs/asrs
 - <imm5> = 0 not allowed
- lsls
 - C unaffected if <imm5> = 0

flags N = result<31>¹
 Z = 1 if result = 0
 Z = 0 otherwise
 C = last bit shifted out
 V unchanged

asrs rd, <rm>, #imm5 T1

rd, rm ∈ { r0..r7 }

imm5 ∈ { 0..31 }

000	10	iiiiii	mmm	ddd
-----	----	--------	-----	-----

rd = extend_sign(rm >> imm5);
 or rd = extend_sign(rd >> imm5);
 (rd = rm / 2^{imm5})

lsls rd, <rm>, #imm5 T1

rd, rm ∈ { r0..r7 }

imm5 ∈ { 0..31 }

000	00	iiiiii	mmm	ddd
-----	----	--------	-----	-----

N Z C

rd = rm << imm5; or rd <<= imm5;

lsrs rd, <rm>, #imm5 t1

rd, rm ∈ { r0..r7 }

imm5 ∈ { 0..31 }

000	01	iiiiii	mmm	ddd
-----	----	--------	-----	-----

N Z C

rd = rm >> imm5; or rd >>= imm5;

Shift / Rotate Instructions

Examples

00000000	4904	ldr	r1,=0xCFFFFFFF
00000002	2203	movs	r2,#3
00000004	4B04	ldr	r3,=0x66666666
00000006	4C05	ldr	r8,=0x99999999
00000008	25E3	movs	r5,#0xE3
0000000A	4111	asrs	r1,r1,r2 ; register
0000000C	111B	asrs	r3,r3,#4 ; immediate
0000000E	41D4	rors	r8,r8,r2 ; register
00000010	00ED	lsls	r5,r5,#3 ; immediate

- What are the values of r1 – r5 after execution?
- What are the values of the flags?

Shift / Rotate Instructions

Examples Shift

- Multiply with 2^n

unsigned / signed

```
lsls r0,r1,#1    ; *2
lsls r0,r1,#2    ; *4
lsls r0,r1,#3    ; *8
```

lsls for signed and unsigned

- Divide by 2^n

unsigned

```
lsrs r0,r1,#1    ; /2
lsrs r0,r1,#2    ; /4
lsrs r0,r1,#3    ; /8
```

signed

```
asrs r0,r1,#1    ; /2
asrs r0,r1,#2    ; /4
asrs r0,r1,#3    ; /8
```

lsrs and asrs differ!

Shift / Rotate Instructions

Multiplication with Constants using lsls and adds

- Example: Multiplication with 13
 - Constant shown as power of 2: $13 = 8 + 4 + 1$
 - $r0 = 13 \cdot r1 \rightarrow r0 = (1 + 4 + 8) \cdot r1 = r1 + 4 \cdot r1 + 8 \cdot r1$

```
movs    r0, r1      ; r0 = r1
lsls    r1, r1, #2   ; 4 • r1
adds    r0, r0, r1   ; r0 = r0 + 4 • r1
lsls    r1, r1, #1   ; 2 • r1 -> 8 • r1
adds    r0, r0, r1   ; r0 = r0 + 8 • r1
```

Conclusion

- Logic Instructions
 - ands, bics, eors, mvns, orrs
- Shift/Rotate Instructions
 - lsls, lsrs, asrs, rors